

# 1 Problema: Instabilità requisiti SW

Molti progetti hanno requisiti **non chiari, instabili e variabili**, numerose prescrizioni da seguire, quantità di documenti e eccessiva rigidità rendono "pesante" lo sviluppo plan-driven. In molti casi è richiesto un approccio flessibile e "leggero".

## 1.1 Successi Processi SW Plan-driven

Approcci plan-driven sono fondamentali per sistemi critici:

- Team numerosi e geograficamente dislocati da coordinare;
- Processi che durano lunghi periodi;
- SW che sarà usato / mantenuto a lungo.

## 1.2 Insuccessi Processi SW Plan-driven

- Modelli plan-driven si adattano poco al contesto dinamico ed alla necessità di consegna rapida di più progetti;
- Eccessivo overhead;
- Poco flessibili a cambiamenti dei requisiti.

# 2 Sviluppo Rapido del SW

Idea: SW deve evolvere rapidamente per riflettere cambiamenti delle necessità dei committenti e utenti finali, deve inoltre essere consegnato presto per essere competitivo sul mercato. **Rapidità di sviluppo e consegna è spesso il requisito più critico per i sistemi SW.** \*inserire immagine\*

# 3 Agilità

Idea: risoste ai cambiamenti, comunicazione fra stakeholder efficaci, portare il cliente nel team di lavoro per avere un feedback rapido → **Agilità consente di avere rapida, incrementale consegna del SW**

- Documentazione minima
  - Focus sul codice, non sulla progettazione;
  - Non vi sono specifiche dettagliate;
  - Overhead di documentazione limitati.
- Consegna rapida / incrementale
  - Sistema sviluppato in incrementi;
  - Stakeholder coinvolti nella specifica valutazione di ogni incremento.
- Strumenti di supporto
  - Utilizzo strumenti al supporto del processo di sviluppo.

## 3.1 Processi Plan-driven e Agili

**Plan driven:** tutte le attività sono pianificate in anticipo, loro avanzamento è misurato rispetto a quanto previsto dal piano. Fasi del processo SW sono distinte tra loro e gli output di ogni fase sono utili per la successiva.

Pratiche plan driven e agili possono coesistere nello stesso processo: processi agili possono produrre documentazione / includere attività pianificate quando necessario, processi plan-driven possono invece essere incrementali. *Per grandi sistemi occorre trovare un compromesso.*

**Agili:** pianificazione incrementale e continua durante sviluppo SW, quindi più facile modificare processo per adeguarsi a modifiche di requisiti del cliente o del prodotto. Requisiti, progettazione e implementazione avvengono assieme.

# 4 Principi Agili

- **Coinvolgimento del cliente:** clienti intervengono ad ogni iterazione del prodotto (valutano e validano iterazione, forniscono eventuali nuovi requisiti / propongono modifiche, assegnano priorità ad ogni requisito richiesto);

- **Accettare cambiamenti:** prodotto non pianificato rigidamente, vanno previsti eventuali cambi dei requisiti del sistema;
- **Mantenere semplicità:** Prodotto e sviluppo devono essere più semplici possibile, possibilmente lavorare attivamente per eliminare complessità del sistema;
- **Sviluppo incrementale:** SW sviluppato e consegnato incrementalmente, cliente specifica requisiti da includere in ciascun incremento;
- **Persone, non processi:** Sviluppo non deve essere fortemente prescrittivo, membri del team devono essere liberi di sviluppare secondo propri metodi in modo da sfruttare al meglio capacità dei singoli.

## 4.1 Applicabilità Metodi Agili

- Prodotti di **piccole / medie dimensioni**;
- Prodotti **personalizzati** con chiaro impegno da parte del cliente nell'essere coinvolto;
- Prodotti con **pochi stakeholder** e regolamenti flessibili;
- Team **fisicamente visini** con comunicazioni informali / facilitate.

## 4.2 Vantaggi sviluppo agile

- Consegne predicibili;
- Rapida risposta a cambiamenti;
- Rischi attenuati;
- Alta produttività;
- Clienti soddisfatti.

# 5 Tecniche Agili

## 5.1 Extreme Programming

Metodo agile più conosciuto, spinge normali pratiche di sviluppo a livelli "estremi". \*Inserire immagine\*

- Approccio iterativo *estremo*;
- Piccoli e frequenti incrementi rilasciati al cliente;
- Requisiti espressi come storie utente (semplici scenari usati come base per decidere che funzionalità va inclusa in un incremento del sistema);
- Scenari divisi in task di sviluppo più semplici, implementati direttamente (no progettazione / documentazione), costituiscono unità principali dell'implementazione;
- Agilità  $\neq$  assenza pianificazione, ma pianificazione flessibile non appesantita da documentazione;
- Programmatori lavorano a coppie (pair programming), sviluppano test per ogni task prima di scrivere il codice (tutti i test devono essere un successo al momento dell'integrazione al sistema);
- Cliente deve essere coinvolto nello sviluppo.

### 5.1.1 EP e Principi Agili

- **Coinvolgimento cliente:** rappresentante del cliente on-site, parte del team di sviluppo;
- **Accettare cambiamenti:** appena un task è concluso viene integrato al sistema;
- **Sviluppo incrementale:** piccoli e frequenti rilasci (aggiunta di funzionalità incrementale);
- **Mantenere semplicità:** progetto più semplice possibile che soddisfi requisiti correnti + costante miglioramento del codice (*refactoring*);
- **Persone, non processi:** programmazione in coppia, proprietà collettiva del codice.

### 5.1.2 Influenza dell'Extreme Programming

EP originale quasi mai adottato in quanto non pratico (richiede cambio radicale del modo di lavorare di un'organizzazione), sue pratiche chiave hanno però ispirato tutti gli approcci agili e vengono implementate nei processi di sviluppo:

- Storie utente;
- Refactoring;
- Sviluppo preceduto dai test;
- Pair programming.

## 5.2 Storie utente

Descrivono requisiti degli utenti come scenari d'uso in cui l'utente potrebbe trovarsi, sono scritti da cliente e team su schede che il team di sviluppo suddivide in **task** da implementare. Task rilevanti sono base per definire pianificazione delle iterazioni e stime dei costi.

### 5.2.1 Pro

- Integrano deduzione dei requisiti con lo sviluppo in modo da gestire i cambiamenti nei requisiti;
- Più semplice relazionarsi con storie utente al posto di un tradizionale documento di requisiti;
- Ordinate in base a quelle che possono fornire supporto utile all'azienda;
- Se requisiti cambiano vengono aggiunte story card e storie non ancora realizzate possono essere modificate / scartate.

### 5.2.2 Contro

- Non semplice stabilire se storie utente coprono completamente requisiti del sistema;
- Clienti esperti potrebbero omettere scenari / task considerati ovvi, che però non lo sono per gli sviluppatori (descrizione incompleta requisito).

## 5.3 Refactoring

Rinuncia a gestire in anticipo cambiamenti, perché si ritiene che modifiche non si possano prevedere con certezza. Propone invece un continuo miglioramento del codice per rendere più semplice implementazione di eventuali modifiche future (mediante riorganizzazione e riscrittura per migliorare efficienza e comprensibilità).

- Team di sviluppo cerca proattivamente aspetti SW da migliorare e implementa miglioramenti immediatamente;
- Miglioramento può riguardare situazioni in cui non vi è necessità immediata;
- Si ritiene che codice di qualità riduca necessità di documentazione e permetta facili modifiche future.

### 5.3.1 Esempi

- Rimozione codice duplicato;
- Rinominare classi, attributi e metodi per renderli più comprensibili;
- Creazione di librerie che contengono codice utile a più classi / progetti.

### 5.3.2 Pro

- Sviluppo incrementale tende a portare a deterioramento del codice: refactoring lo mitiga migliorando struttura e leggibilità;
- Esistenza di tool per automatizzazione alcune operazioni di refactoring.

### 5.3.3 Contro

- Refactoring a livello di codice non basta per supportare un cambiamento, necessaria modifica dell'intera architettura (più costosa);
- Bisogna trovare un compromesso tra tempo dedicato a sviluppo di nuove funzionalità e refactoring.

## 6 Tecniche Agili - Sviluppo Preceduto dai Test

In EP il SW è testato dopo ciascun cambiamento:

- **Sviluppo test driven:** casi di test da soddisfare scritti prima del codice → *guidano lo sviluppo*;
- **Automatizzazione Test:** strumenti eseguono automaticamente tutti i test a ogni nuova versione;
- **Coinvolgimento cliente:** cliente coinvolto nello sviluppo test d'accettazione delle storie da implementare nell'iterazione successiva.

Idea: scrivere test prima del codice chiarisce requisiti da implementare (aspetto cruciale in assenza di specifiche documentate tipo *plan-driven*). Test sono scritti come programmi per essere eseguiti automaticamente, simulano input e leggono output (è possibile esecuzione mentre si programma per fare hotfix). **Lo sviluppo non può procedere fino al superamento di tutti i test.**

### 6.1 Automazione Test

- Test scritti come programmi eseguibili, non insismi di dati;
- Programma di test simula immissione di un determinato input (valuta se output ottenuto corrisponde a quello atteso);
- Framework come JUnit facilitano scrittura / esecuzione test;
- Test eseguiti a ogni aggiunta di funzionalità;
- Automatizzazione rende più facile e rapida fase di verifica / validazione.

### 6.2 Pro

Scrittura test implica definizione di un'interfaccia e una specifica comportamentale della funzionalità da sviluppare (riduzione possibili incomprensioni / ambiguità / omissioni).

### 6.3 Contro

- Pratica onerosa per il cliente;
- Sforzo per tenere aggiornati test interessati alle modifiche del codice;
- Test potrebbero essere incompleti (per varie ragioni non verificano tutti i possibili scenari).

## 7 Tecniche Agili - Pair Programming

Idea: programmatori lavorano in coppie alla stessa postazione (coppie variano continuamente), refactoring viene incoraggiato in quanto spesso benefico per il team. Il SW è di proprietà dell'intero team, i singoli non sono ritenuti responsabili degli errori nel codice (*egoless programming*), è il team che ha responsabilità collettiva della risoluzione dei problemi.

### 7.1 Pro

- Aiuta a sviluppare senso di proprietà del codice nel team / diffondere conoscenza nell'ambito del team;
- Permette processo di revisione informale (sviluppatori verificano reciprocamente proprio lavoro);
- Riduce rischi di fallimento dovuti a turn-over.

### 7.2 Contro

Talvolta può essere meno efficiente della programmazione individuale, soprattutto in caso di programmatori esperti.

## 8 Gestione Agile - Scrum

Metodo agile Scrum (Schwaber e Beedle 2001; Rubin 2013) offre framework per organizzare agilmente progetti / fornire in una certa misura una visibilità esterna su ciò che sta accadendo all'interno del team di sviluppo SW. \*Inserisci Immagine\*

- Ogni sprint parte da **product backlog**, la lista degli elementi (caratteristiche prodotto, requisiti e miglioramento ingegnerizzazione);

- Versione iniziale del backlog può derivare da un documento dei requisiti, da una lista di storie utente. *Backlog non riguarda solo codice, ma anche architettura / documentazione*;
- All'inizio di ogni ciclo product owner stabilisce priorità del backlog per definire elementi più importanti;
- Tutti i membri del team coinvolti nella scelta degli elementi con priorità alta che dovranno essere completati, valutano tempo richiesto per il completamento dei requisiti in base alla **velocità** (stimata su quella degli sprint precedenti per capire quanto lavoro si riuscirà a svolgere);
- Pianificazione dello sprint porta a **sprint backlog** (lavoro da svolgere in quello sprint);
- Team (~7 persone) sceglie chi dovrà lavorare su determinati elementi ed avvia lo sprint(~2-4 settimane);
- Riunioni giornaliere (breve incontro faccia a faccia) del team esaminano avanzamento del lavoro / stabilisce priorità del giorno. ScrumMaster deve garantire che processo Scrum sia eseguito efficientemente, solitamente potrebbe essere un Project Manager;
- Incremento del SW consegnato da uno sprint dovrebbe essere rilasciabile (in uno stato finito senza bisogno di ulteriore lavoro per incorporarlo nel prodotto finale): nella pratica non è sempre realizzabile.
- Al termine di ogni sprint si tiene riunione di verifica con tutti i membri del team per migliorare il processo ed avere input su prodotto e sul suo stato per revisione dei backlog.